изучи документацию по панели 3x-ui и напиши документацию по api запросам для нее

Ниже — практическая документация по API 3x-ui на основе официальной wiki, исходного кода контроллера и модели данных панели. Основной API для управления inbound'ами находится под префиксом `/panel/api/inbounds` и требует авторизацию через логин-сессию.[1] [2] [3]

Авторизация

Перед вызовом API нужно войти в панель через `POST /login` с телом:

```
{"username":"...","password":"..."}
```

После успешного входа запросы к API выполняются в той же сессии, обычно с cookie сессионного идентификатора.[1]
Если панель стоит за reverse proxy, важно не потерять cookie и проброс заголовков, иначе авторизация сломается.[2]

Базовый путь

Все методы ниже находятся под:

```
/panel/api/inbounds
```

То есть полный путь формируется как:

```
https://host:port/panel/api/inbounds/<endpoint>
```

Например, список inbound'ов: `GET /panel/api/inbounds/list`.[3] [1]

Методы inbound

Метод	Путь	Назначение
GET	`/list`	Получить все inbound'ы [1] [3].
GET	`/get/:id`	Получить inbound по `id` [1] [3].
GET	`/getClientTraffics/:email`	Получить трафик клиента по email [1] [3].
GET	`/getClientTrafficsById/:id`	Получить трафик клиента по inbound id [1] [3].

Метод	Путь	Назначение
GET	`/createbackup`	Отправить backup админам через Telegram bot [1] [3].
POST	`/add`	Добавить inbound [1] [3].
POST	`/del/:id`	Удалить inbound [1] [3].
POST	`/update/:id`	Обновить inbound [1] [3].
POST	`/clientIps/:email`	Получить IP-адреса клиента [1] [3].
POST	`/clearClientIps/:email`	Очистить IP клиента [1] [3].
POST	`/addClient`	Добавить клиента в inbound [1] [3].
POST	`/:id/delClient/:clientId`	Удалить клиента [1] [3].
POST	`/updateClient/:clientId`	Обновить клиента [1] [3].
POST	`/:id/resetClientTraffic/:email`	Сбросить трафик клиента [1] [3].
POST	`/resetAllTraffics`	Сбросить трафик всех inbound'ов [1] [3].
POST	`/resetAllClientTraffics/:id`	Сбросить трафик всех клиентов в inbound [1] [3].
POST	`/delDepletedClients/:id`	Удалить истощённых клиентов; `-1` означает всех [1].
POST	`/onlines`	Получить список онлайн-пользователей по email [1] [3].

Формат данных inbound

Модель inbound включает поля вроде `id`, `up`, `down`, `total`, `remark`, `enable`, `expiryTime`, `trafficReset`, `listen`, `port`, `protocol`, `settings`, `streamSettings`, `tag`, `sniffing`.[4]
Практически это значит, что при `add` и `update` вы передаёте не только метаданные inbound'а, но и Xray-конфигурацию: порт, протокол, параметры stream и JSON-строку настроек.[4]
Поле `settings` обычно хранит JSON с клиентами и параметрами протокола, а не отдельную структуру в виде нескольких независимых полей.[5] [4]

Формат данных клиента

Модель клиента включает такие поля, как `id`, `password`, `flow`, `email`, `limitIp`, `totalGB`, `expiryTime`, `enable`, `tgId`, `subId`, `comment`, `reset`.[4]
Для VLESS/VMess `clientId` в удалении и обновлении обычно соответствует `client.id`; для TROJAN — `client.password`; для Shadowsocks — `client.email`.[1]
Это важная деталь, потому что одинаковый API-метод использует разный идентификатор клиента в зависимости от протокола.[1]

Примеры запросов

Получить список inbound'ов

```
curl -s -X GET "https://host:2053/panel/api/inbounds/list" \
  -H "Cookie: _xui_session=YOUR_SESSION"
```

Получить конкретный inbound

```
curl -s -X GET "https://host:2053/panel/api/inbounds/get/1" \
  -H "Cookie: _xui_session=YOUR_SESSION"
```

Добавить клиента

Из открытых примеров SDK видно, что добавление клиента отправляет `id` inbound'а и строку `settings`, где параметры клиента сериализуются в JSON.[6] [5]
Пример:

```
curl -s -X POST "https://host:2053/panel/api/inbounds/addClient" \
  -H "Cookie: _xui_session=YOUR_SESSION" \
  -H "Content-Type: application/json" \
  -d '{
    "id": 1,
    "settings": "{\"clients\":[{\"id\":\"95e4e7bb-7796-47e7-e8a7-f4055194f776\",\"ema
  }'
```

Обновить клиента

Для `updateClient/:clientId` используется похожий формат, а публичные SDK и Postman-документация подтверждают, что это отдельный endpoint для редактирования клиента.[7] [8]
Обычно в тело также передают `id` inbound'а и `settings` в сериализованном виде.[5]

Полезные замечания

Для панели есть публичная Postman-коллекция и документация, где можно взять готовые запросы и переменные окружения вроде `HOST`, `PORT`, `WEBBASEPATH`, `username`, `password`.[9] [8]
В репозитории также есть Node.js SDK, который подтверждает, что API панелью реально используется програмно и удобен для создания, редактирования и удаления inbound'ов и клиентов.[10] [11]
Если нужен именно машинно-ориентированный слой, то лучше опираться на эти endpoint'ы и на фактические структуры `Inbound` и `Client`, а не на UI-логику панели.[3] [4]

Что я могу сделать дальше

Могу сразу подготовить для тебя:

- полноценную Markdown-**документацию** по API 3x-ui;
- OpenAPI/Swagger YAML;
- готовую Postman collection;

- или Python-**клиент** для работы с этими запросами.

⁂

1. https://github.com/MHSanaei/3x-ui/wiki/Configuration
2. https://github.com/MHSanaei/3x-ui/issues/3207
3. https://github.com/MHSanaei/3x-ui/blob/main/web/controller/api.go
4. https://pkg.go.dev/github.com/mhsanaei/3x-ui/v2/database/model
5. https://pkg.go.dev/github.com/digilolnet/client3xui
6. https://app.unpkg.com/3x-ui@1.6.0/files/examples/add-client.ts
7. https://www.postman.com/hsanaei/3x-ui/request/fts6hq9/update-client
8. https://www.postman.com/hsanaei/3x-ui/documentation/q1l5l0u/3x-ui
9. https://www.postman.com/hsanaei/3x-ui/collection/q1l5l0u/3x-ui
10. https://github.com/mehdikhody/3x-ui-js
11. https://github.com/iamhelitha/3xui-api-client
12. https://github.com/MHSanaei/3x-ui/wiki/Configuration/097b575ab088b469874d586349a091604aa1d183
13. https://pkg.go.dev/github.com/iprodev/3x-ui/v2
14. https://github.com/MHSanaei/3x-ui
15. https://wiki.senko.digital/vpn/3x-ui
16. https://github.com/MHSanaei/3x-ui/issues/3052
17. https://3x-ui.com/how-to-update-3x-ui/
18. https://github.com/MHSanaei/3x-ui/issues/3244
19. https://github.com/MHSanaei/3x-ui/issues/3354
20. https://pkg.go.dev/github.com/nextster/client3xui